Zamba: A Compact 7B SSM Hybrid Model

Paolo Glorioso Quentin Anthony Yury Tokpanov James Whittington Jonathan Pilault
Adam Ibrahim Beren Millidge
{paolo, quentin, yury, james, jonathan, adam, beren}@zyphra.com

Zyphra
Palo Alto, CA

arXiv:2405.16712v1 [cs.LG] 26 May 2024

***Abstract*—In this technical report, we present Zamba, a novel 7B SSM-transformer hybrid model which achieves competitive performance against leading open-weight models at a comparable scale. Zamba is trained on 1T tokens from openly available datasets and is the best non-transformer model at this scale. Zamba pioneers a unique architecture combining a Mamba backbone with a single shared attention module, thus obtaining the benefits of attention at minimal parameter cost. Due to its architecture, Zamba is significantly faster at inference than comparable transformer models and requires substantially less memory for generation of long sequences. Zamba is pretrained in two phases: the first phase is based on existing web datasets, while the second one consists of annealing the model over high-quality instruct and synthetic datasets, and is characterized by a rapid learning rate decay. We open-source the weights and all checkpoints for Zamba, through both phase 1 and annealing phases.**

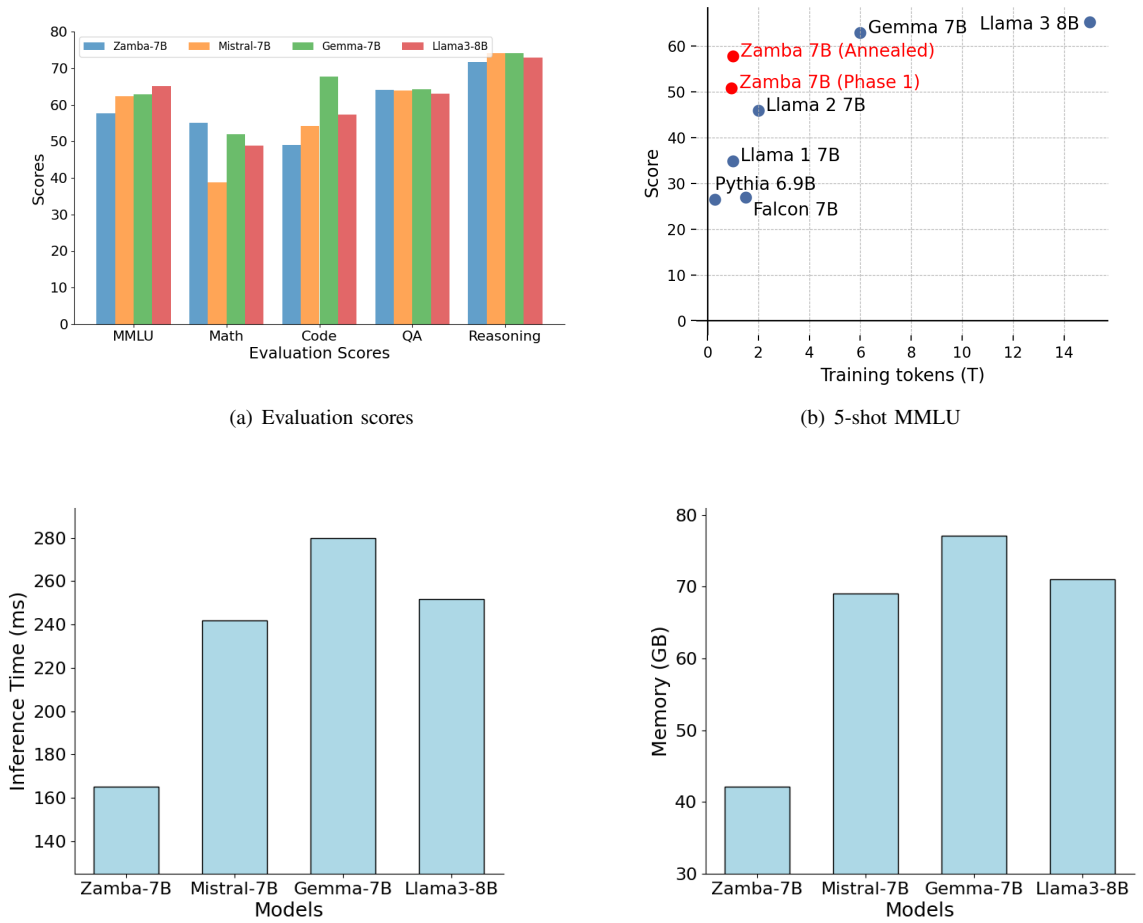
I. Introduction

The transformer architecture (Vaswani et al., 2017) has revolutionized natural language processing and many other fields of deep learning (Dosovitskiy et al., 2020; Yang et al., 2023; Touvron et al., 2023) thanks to its ability to scale across model size, dataset size and compute (Kaplan et al., 2020; Brown et al., 2020; Hoffmann et al., 2022; Rae et al., 2022). Nevertheless, it is unclear whether transformers are the only architecture able to leverage scale (Bachmann et al., 2023), and new architectures might prove to have even greater scaling potential. Moreover, the quadratic cost of the core attention computation in transformers remains an important limitation and bottleneck of the architecture. This caveat has led to a search for architectures circumventing this bottleneck while maintaining the performance and scaling capabilities of transformers.

One promising line of research has been in state-space models (SSMs) (Gu et al., 2021a), which replaces the attention operation in transformers with a linear dynamical system which can be computed in two ways – either as a recurrent dynamical system or as a parallel scan. This parallel mode ensures efficient training on GPU hardware, while the recurrent mode which enables linear-time and constant-memory generation, similar to a recurrent neural network (RNN). Because the linear dynamics maintain only a fixed-size hidden state, inference time in these systems is not quadratic in the context length, and does not require a linearly growing KV cache, rendering SSM architectures constant in memory during generation. Potentially, both of these advantages could lead to significantly longer contexts being feasible for SSMs as opposed to transformers.

While early SSM models performed significantly worse than transformers on language tasks, recent SSMs such as Mamba (Gu and Dao, 2023) or Griffin (De et al., 2024) appear to be closing the gap. Such models rely on input-dependent dynamics for the SSM sequence mixer, analogous to the ways attention uses input-dependent Q, K,V matrices. In spite of these improvements, Jelassi et al. (2024) find that they do not fully match the expressivity and performance of transformers at scale, with several works highlighting in particular in-context learning (ICL) weaknesses (Park et al., 2024; Grazzi et al., 2024). Recent works have argued that combining Mamba and attention blocks can lead to improved performance, potentially matching that of full attention (i.e., a pure transformer). In their detailed study of various possible hybrid architectures, Poli et al. (2024) find that approximately one quarter of the layers being self-attention and the rest SSMs was optimal.

Another approach to improving language model efficiency has been Mixture-of-Experts (MoE) architectures (Shazeer et al., 2016; Fedus et al., 2022; Rajbhandari et al., 2022), which can be combined with SSMs (Anthony et al., 2024; Lieber et al., 2024). MoE models consist in routing the input to subsets of parameters at specific layers, based on the input to these layers (Jacobs et al., 1991). In transformers, this routing strategy is typically applied to the feed-forward layers (or MLPs) following attention blocks. The intuition being that this routing allows parts of the model to specialize in handling different situations, making it unnecessary to activate all the parameters in the model at all times to achieve good performance. MoEs thus trade increased parameter count—and hence memory cost—for reduced FLOPs during training and at inference. This trade-off is very appealing when serving large language models (LLMs) at scale, contributing to the popularity of MoE architectures. However, in other use cases, e.g. running LLMs on local devices with limited memory such as consumer GPUs, loading the model in memory can become a greater bottleneck than the FLOPs of forward passes. This reverses the trade-off, and it becomes interesting to explore strategies to boost performance using additional FLOPs instead of extra parameters.



(a) Evaluation scores

(b) 5-shot MMLU

(c) Forward-pass latency at 8k sequence length

(d) Memory usage for generation

Fig. 1. Evaluation of academic benchmarks, inference and generation performance for Zamba vs other leading models at this scale. We observe that Zamba approaches but is slightly behind the performance of such models at many evaluations, however, as panel (b) shows, it has been trained on much fewer tokens. Due to its architectural innovations, it can be inferenced significantly faster and requires much less memory for generation. Evaluation results: MMLU is 5-shot, Math is GSM8k 5-shot, code is Human-Eval pass@100, QA is mean of zero-shot BoolQ and OpenBookQA, reasoning is mean of zero-shot ARC, HellaSwag, WinoGrande, and PIQA.

Several recent works have begun to openly explore curriculum learning approaches (Elman, 1993; Bengio et al., 2009) to training large language models. Liu et al. (2018) show that a curriculum of increasing quality of data leads to improvements on question answering tasks. Krishna et al. (2023) report that finetuning data appears to be beneficial during pretraining. Gunasekar et al. (2023) demonstrates that continual pretraining with high-quality synthetic data can significantly boost model's performance. Gupta et al. (2023) and Ibrahim et al. (2024) highlight how in continual pretraining settings, strategies with the learning rate schedule and replay lead to improved adaptation on training data, while mitigating forgetting on previously learnt data. The intuition behind Zamba's curriculum approach is that (1) finetuning and synthetic data may be considered to have higher quality than web data, and (2) while high-quality data is much less available than general web data, the fast adaptation in few training steps shown with the last phase of infinite schedules of Ibrahim et al. (2024) allows leveraging the high-quality data in relatively few iterations at the end of training. This approach has also been hinted at by current frontier models (Gemma Team et al., 2024; Team et al., 2023) or Abdin et al. (2024) as a major method of improving performance. During the development of Zamba, many recent works have begun reporting a two-phase curriculum approach, which utilizes standard pretraining web-data for the initial pass followed by an 'annealing phase' in which the model is trained on high quality instruct and synthetic data during a rapid learning rate decay. The intuition is that this rapid decay helps the model focus more on assimilating the newer higher quality data than if it was just included throughout all of pretraining, and is especially useful when the amount of high quality data available is small relative to the bulk pretraining data. The first open description of this form of training was miniCPM (Hu et al., 2024), who describe the method in some detail, followed by Nemotron (Parmar et al., 2024a) and OLMo-v1.7 (AI2, 2024).

A. Overview

In this technical report, we release and describe the training process for *Zamba*, a 7B Mamba-based SSM with a novel global shared attention architecture. Zamba was trained on only 1T tokens of open web datasets, yet its performance approaches that of the leading ∼7B transformer-based models (see Figure 1(a)). Zamba's unique architecture combines a Mamba backbone with a global shared self-attention layer (see Figure 2), merging the benefits of transformers for retrieval and in-context learning with the inference efficiency of Mamba. This architecture provides a way to boost performance at a small and constant parameter and hence memory cost. Zamba is also the highest-performing SSM in the small 7B model range and the highest-performing dense SSM model available. Zamba matches state-of-the-art 7B models on many linguistic evals, while lagging slightly behind on tests of reasoning and in context learning, which may be due to the significant data disparity between Zamba and other leading ∼7B models.

Our novel architecture was inspired by the relation between the cortex and hippocampus in the brain (Whittington et al., 2020, 2021) – where different layers and regions of the cortex, despite performing different tasks, all send and receive information from a single shared memory store in the hippocampus. Additionally, our core concept of shared layers allowing us to spend additional flops to improve performance is closely related to work on recurrent models (Dehghani et al., 2018) which share deep similarities to neural processing (van Bergen and Kriegeskorte, 2020; Tscshantz et al., 2023). While still early, we believe that the Zamba architecture is an important step towards designing architectures with a global and shared memory store similar to how the brain operates.

We train Zamba in a two-phase approach utilizing a general pretraining and annealing phase on high-quality datasets coupled with rapid learning rate decay. Like Ibrahim et al. (2024) and in contrast to Hu et al. (2024), we find it better to rewarm the learning rate followed by a rapid exponential (instead of linear) decay. We find that this significantly improves the performance of our model upon some downstream evaluations, but has little effect on others. To aid open study of the effects of this annealing process, we release both the final annealed model and the model after only phase-1 training. Additionally, we release all checkpoints during training for both phase 1 and the annealing phase. Zamba is the most performant open checkpoints model, surpassing the concurrently released OLMo 1.7 (AI2, 2024) as well as significantly outperforming Pythia (Biderman et al., 2023). We believe that open checkpoints are crucial for allowing the academic community access to key data around state-of-the-art model training and to enable open and productive study of key questions relating to learning dynamics. Our model is also the only open-checkpoints SSM of any significant scale or performance and we hope that our work may be useful for researchers to understand more deeply how learning works in these alternative architectures compared to transformers.

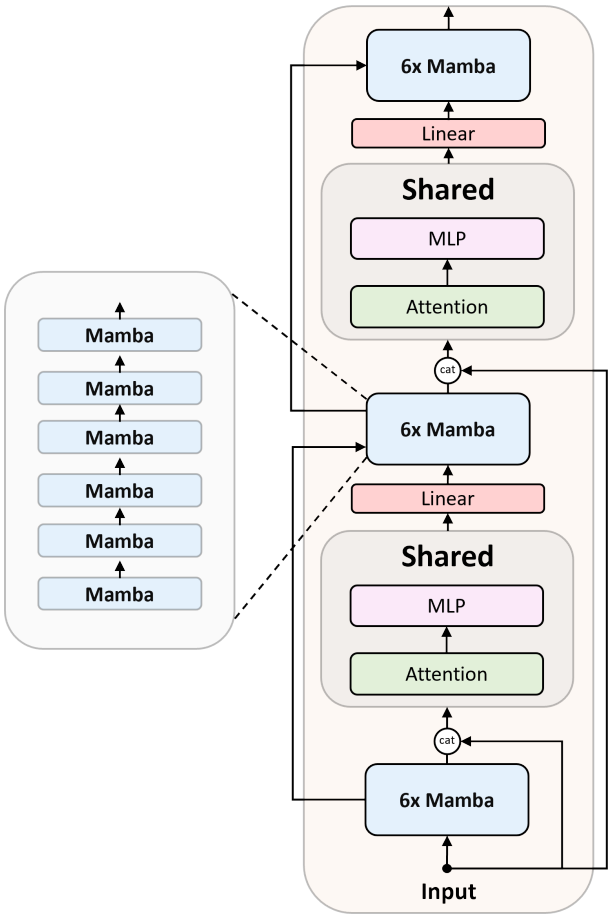


Fig. 2. The Zamba architecture. Zamba consists of a backbone of standard Mamba blocks connected to a shared attention and MLP block. This block is repeated every 6 Mamba blocks but has shared parameters, which enables Mamba to utilize more FLOPs for increased performance at the same memory cost. The input embeddings are always concatenated with the residual stream going into the shared attention block as this provides an additional path for the model to remember the inputs. After the block, a learnt linear projection maps the output back to the residual stream.

B. Contributions

In sum, our contributions with Zamba are as follows:

- State-of-the-art Transformer-SSM hybrid architecture at 7B scale, that preserves the FLOP-efficiency of SSM as well as the in-context learning ability of attention
- Novel neuroscience-inspired shared-attention optimization that preserves the modeling performance of independent attention blocks, while saving memory
- Successful replication of two-phase training methods in a large-scale model

The phase 1 and annealing checkpoints of Zamba can be downloaded from: https://huggingface.co/Zyphra/Zamba-7B-v1-phase1 and https://huggingface.co/Zyphra/Zamba-7B-v1.

II. Model

Transformer architectures are typically constructed by alternating blocks of self-attention, which perform sequence mixing, and MLPs, which perform per-token processing. These

blocks are arranged around a residual stream, which theoretically enables faithful signal propagation through the network, and the outputs from the residual stream are normalized by a layer-norm layer. A schematic mathematical representation of a single transformer layer is as follows:

$$x_{l+1} = x_l + \text{MLP}(\text{LN}(\text{Self-Attention}(\text{LN}(x_l))) \quad (1)$$

where x_l represents the activations of the residual stream at layer l. The computation and memory requirements and the self-attention block are linear and quadratic in the sequence length, respectively, thus motivating the search for alternative architectures.

By contrast, SSM architectures such as Mamba typically collapse the sequence mixing and token processing together into a single block, thus resulting in an entirely uniform architecture. Like a transformer, Mamba also uses a residual stream gated by a layer norm to provide the inputs for each mamba block, resulting in the following schematic:

$$x_{l+1} = x_l + \text{Mamba}(\text{LN}(x_l)) \quad (2)$$

Like all SSMs, the sequence mixer of the Mamba block is based around a core linear dynamical system. The key innovation of Mamba is to make the control and observation matrices, as well as the timestep of this system, input-dependent. That is, the core dynamics of the Mamba SSM are, schematically,

$$\begin{aligned} h_{t+1} &= \exp(A\delta_t)h_t + B_t x_t \\ y_t &= C_t h_{t+1} \end{aligned} \quad (3)$$

where x_t is the input to the system at each sequence element, h_t is the internal 'memory' state of the Mamba block, y_t is the output of the sequence mixer, δ_t is an input-dependent timestep arising from the discretization of the continuous time dynamics, and B_t and C_t are input-dependent matrices, while A is an input-independent matrix.

The Mamba block performs both sequence mixing through the core SSM components as well as token processing via projection layers and a gating unit. Inputs to the Mamba block are first processed by a linear layer which splits apart the input into a vector to be processed and a gating vector. This is followed by a 1d convolution which performs some preliminary sequence mixing, the output of which is then used to create the input-dependent B_t,C_t and δ_t matrices, followed by the SSM sequence mixer as in Equation 3. The output y_t of the sequence mixer is then elementwise multiplied by the gating vector from the input, before being passed through a final linear layer to the output. Schematically,

$$\text{Mamba}(x) = \text{Lin}(\sigma(\text{Lin}(x)) \odot \text{SSM}(\text{Conv1D}(\text{Lin}(x)))) \quad (4)$$

Due to the gating and the linear projections, the Mamba layer can perform token processing as is performed by the MLPs in the transformer architecture, while also containing the SSM sequence mixer. This means that in a Mamba vs Transformer architecture of approximately the same size, the Mamba model will have twice the sequence mixers, although potentially they are less expressive than full self-attention.

Zamba consists of a pure Mamba backbone augmented with an additional global shared self-attention (GSA) block every N Mamba blocks. The GSA block consists of a self-attention block and MLP block in series, with shared weights for both. This means that although attention is performed many times throughout the network, it uses the same parameters and thus reduces the memory cost of the model both for the attention parameters and, crucially, for the KV cache size during generation. To allow for some specificity for each GSA call, there is an un-shared learnable linear mapping from the GSA block to the residual stream. Additionally, each input to the GSA block is the concatenation between the residual activations at that layer with the initial residual activities (i.e., after the initial input embedding). This was done to ensure that information from the start of the network is always available to the self-attention. We find that this gives a consistent improvement in performance, perhaps indicating that the residual stream does not maintain information as faithfully throughout the network as the network requires. Due to concatenation, query, key and value vectors in the GSA block have double dimension, which is then projected back, prior to the MLP block, to the residual dimension through the attention's out projector. Schematically, the Zamba GSA block can be represented as,

$$y_l = \text{Lin}_l(\text{MLP}(\text{LN}(\text{Self-Attention}(\text{LN}([x_l, x_0]))))) \quad (5)$$

every N blocks and otherwise pure Mamba. The subsequent Mamba layer is then computed by adding y_l to the non-residual part of the input:

$$x_{l+1} = x_l + \text{Mamba}(\text{LN}(x_l + y_l)) \quad (6)$$

In early ablation studies on small models, we found that this architecture gave a consistent improvement over pure Mamba and also significantly beat parameter-matched Mambaformer (Park et al., 2024), or hybrid architectures, which combine self-attention and Mamba blocks. As such, the success of the Zamba architecture provides evidence that perhaps many of the parameters of self-attention layers are redundant and that they can be replaced with a single, repeated, self-attention operation. We believe that, given that often the primary bottlenecks for running models is the memory cost of the model both in number of parameters and generation, it is valuable to investigate architectures which trade additional inference FLOPs for reduced parameters while maintaining performance.

A. *Phase 1*

We trained Zamba-v1 in two phases. First, we performed a standard pretraining phase on open web datasets. During this phase, Zamba-v1 was trained on approximately 950 Billion tokens. We performed a cosine decay from an initial learning rate of $1.5 * 10^{-4}$ to $7.5 * 10^{-5}$; this decay is significantly less steep than is common, to ensure that there is room for the rapid decay in the annealing phase. Zamba-v1 was trained on 128 H100 GPUs for approximately 30 days. We used data-parallelism and tensor-parallelism across two ranks, performed across attention, MLP and the Mamba blocks. We

used the Zero-1 distributed optimizer to shard optimizer states across ranks and activation checkpointing. We trained with a sequence length of 4096 tokens, using the Mistral tokenizer. A full set of training hyperparameters can be found in Appendix A.

B. Datasets

Our phase 1 dataset consists of a mixture of open web datasets and included The Pile (Gao et al., 2020), RefinedWeb (Penedo et al., 2023), C4 (Raffel et al., 2020), PeS2o (Soldaini and Lo, 2023), and arxiv. The composition of our dataset is shown in Table I.

Dataset Name	Proportion
peS2o	4.89%
C4	12.85%
Pile	15.48%
RefinedWeb	62.35%
Cosmopedia	3.55%
arxiv	0.90%

TABLE I
PROPORTION OF TOKENS OF SUB-DATASETS MAKING UP OUR FULL 1T PRETRAINING DATASET. ALL DATASETS WE USE ARE OPEN AND AVAILABLE.

We performed relatively minor filtering and then LSH-based fuzzy deduplication on this dataset.

We applied the following filters to Pile, and C4-en components of our datasets: minimum length of 100 characters, minimum mean word length of 3 characters, maximum mean word length of 12 characters, maximum fraction of non-alphanumeric characters of 0.3, maximum fraction of numeric characters of 0.2. For fuzzy deduplication we computed min-hashes with signature size of 128, computed on 13-grams based on words. This filtering removed roughly 10% of Pile-dedup and 0.5% of C4-en.

We deduplicated each dataset both against itself and against the other datasets in our full dataset. We built the LSH index targeting 50% Jaccard similarity threshold (25 bands with the range of 5) by inserting the following datasets in the particular order: first Pile-dedup, then C4-en, peS2o, arxiv_s2orc_parsed. We did not insert RefinedWeb into the index, but simply ran its every document through it (RefinedWeb is extensively deduplicated by their authors, so we decided not to spend time to deduplicate is against itself, but only against other datasets). The percentage of identified duplicates in every dataset can be found in Table II. We did not apply any processing to Cosmopedia, since we reasoned its synthetic nature should imply it is mostly high quality and not duplicated in other datasets.

We did not upsample or perform multiple epochs over any components of our phase 1 dataset. Our total dataset consisted of just over 1 trillion tokens, of which the pretraining phase utilized 950 billion tokens.

Dataset Name	Percentage of duplicates
Pile	25%
C4	30%
peS2o	31%
arxiv	7.1%
RefinedWeb	1.2%

TABLE II
PERCENTAGE OF DOCUMENTS MARKED AS DUPLICATES BY FUZZY LSH-MINHASH DEDUPLICATION.

C. Annealing

The annealing phase follows the intuition from the infinite learning rate schedules presented in Ibrahim et al. (2024), which shows quick improvements thanks to an exponential decay. Doing this on higher-quality data allows curriculum approaches to require much less high-quality tokens than usual during pretraining. Similarly to miniCPM (Hu et al., 2024) and Nemotron (Parmar et al., 2024a), we performed a rapid LR decay over a dataset consisting of a mixture of high-quality tokens and our pretraining dataset. We used a replay fraction of 60% original pretraining data and 40% new annealing datasets. Our annealing dataset comprised a large collection (over 100) of existing high-quality datasets from various sources. These include large math (such as StackMathQA) and code datasets (such as EvolInstructCode), as well as instruct finetuning (such as OpenOrca) datasets, and some synthetic data examples from more powerful language models. For most of these datasets, we performed only one epoch, but for select high-quality subsets, we performed 2 epochs. We utilized an exponential decay schedule of the form $\eta_t = Ae^{-t/(\gamma T)} + B$, where t is the iteration number, T is the total annealing iterations, and we found $\gamma = 0.25$ to be the optimal value. The coefficients A and B are found by requiring η_0 and η_T to be the learning rate after warmup and at the end of annealing, respectively, where we used $\eta_0 = 1.1 * 10^{-4}$ and $\eta_T = 10^{-7}$. Unlike miniCPM, we found that rewarming the learning rate from 0 back to the maximum learning rate and then decaying again performed better than beginning the annealing phase at the final learning rate of the original pretraining run – a similar finding to Ibrahim et al. (2024) in the continual pretraining setting. We additionally found that our exponential decay schedule outperformed the linear decay used in miniCPM. Finally, while we did see a decrease in loss on the original pretraining dataset, it was not as pronounced as described by miniCPM, which we ascribe to Zamba being a much larger model trained on more tokens, or could potentially highlight differences in our annealing datasets or schedule.

III. PERFORMANCE

We evaluate Zamba on a suite of standard language modelling evaluation benchmarks including reasoning, question-answering, math, code, and general knowledge questions. In general, the trend is that Zamba performs well for its size, and is generally far ahead of competing models trained on

Model	PIQA	ARC-Easy	ARC-Challenge	BoolQ	WinoGrande	HellaSwag	OpenBookQA	MMLU	Tokens
Llama 3 8B	80.96	77.61	53.58	81.16	73.24	79.13	45.0	**65.17**	15 T
Mistral 7B v0.1	**82.26**	79.59	53.92	**83.64**	73.88	**81.07**	44.2	62.30	?
Gemma 7B	81.12	**80.77**	**54.27**	83.12	73.8	80.46	**45.2**	62.85	6 T
Pythia 6.9B	76.93	63.4	35.9	64.2	62.43	65.8	38.8	26.51	0.3 T
Falcon 7B	80.74	70.88	43.69	73.79	67.25	76.39	43.6	26.90	1.5 T
Llama 2 7B	79.05	74.58	46.16	77.68	69.06	75.97	44.2	45.88	2 T
Llama 1 7B	79.16	72.94	44.71	75.11	70.01	76.24	44.6	34.88	1 T
OLMo 7B 1.0	79.43	68.69	40.36	72.48	66.37	75.67	42.2	N/A*	2.5 T
Zamba 7B (Phase 1)	79	70.74	43.9	77.5	70.8	77.14	44.2	50.82	0.95 T
Zamba 7B (Annealed)	81.37	74.5	46.48	83.6	**76.4**	80.24	44.6	57.72	1 T

TABLE III

ZERO-SHOT EVALUATION RESULTS ON LANGUAGE MODELLING BENCHMARKS, EXCEPT FOR MMLU WHICH IS FIVE-SHOT, COMPARING ZAMBA WITH OPEN-WEIGHT MODELS OF A COMPARABLE PARAMETER COUNT. WE ALSO REPORT THE TRAINING TOKEN COUNT (IF KNOWN) TO DEMONSTRATE THE SAMPLE EFFICIENCY OF ZAMBA. FIRST PLACE IS **BOLDED**, SECOND PLACE IS UNDERLINED.

open datasets (see Figure 3), but slightly lags behind the performance of the leading models (see Figure 1). However, per-token (and hence per training FLOP), Zamba is extremely efficient relative to comparable models. We believe that this is primarily due to the fact that Zamba has been trained on significantly fewer tokens than leading models, and potentially using lower quality open-web data compared to the secret, in-house datasets of the large AI companies. Nevertheless, we note that Zamba outperforms Llama2 and hence is not a weak model. Zamba's performance on code is relatively poor compared to leading models and we believe that this is most likely explained by the relative lack of code in our dataset, since we used only direct web datasets and not github. Zamba appears to perform well, often competitively with the leading models on general language modeling and reasoning benchmarks such as PIQA, Winogrande, and HellaSwag, which is interesting since it has likely been trained on substantially fewer and potentially lower-quality tokens than these models, and this may be indicative of the architectural benefits of Zamba.

In terms of inference and generation efficiency, Zamba is extremely performant. Despite utilizing more FLOPs per parameter due to our parameter sharing, Zamba's forward pass is significantly faster than competing models at the 7B scale, an advantage which increases with longer sequence lengths. Additionally, due to Zamba's SSM backbone, memory required for KV caching in Mamba is reduced by a large factor compared to other models of a similar scale, thus enabling Zamba to generate more efficiently and achieve significantly longer contexts on a single device (see Figure 4.

Zamba possesses a number of architectural qualities that make it particularly amenable to inference efficiency:

- Zamba's sparing use of attention layers keeps its KV-cache memory particularly low. Specifically, there is a single attention block applied 13 times within Zamba-7B, with independent activations and KV-cache entries at each invocation.
- Mamba blocks have a significantly higher throughput than comparable attention or MLP blocks (Gu and Dao, 2023).
- We apply the most efficient available kernel for each available block. Mamba kernels were taken from the open-source mamba implementation (Gu and Dao, 2023) and tuned for the H100 architecture. Attention is implemented via the Flash Attention v2 kernels (Dao, 2023). The RMSNorms are implemented from the Transformer Engine library (NVIDIA, 2023).

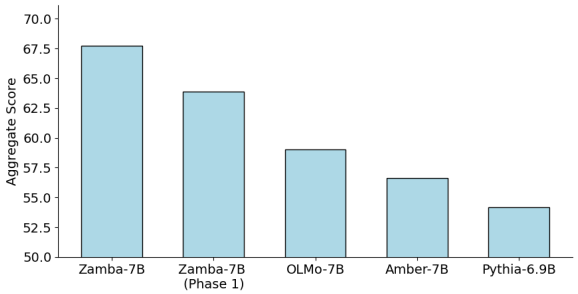


Fig. 3. Aggregated evaluation performance of Zamba vs alternative open-checkpoint models Olmo 1.0, Amber-7B and Pythia. We observe that Zamba both phase 1 and annealed models significantly outperform competing open-checkpoint models. The aggregate eval is the mean score of PIQA, ARC-Easy, ARC-Challenge, HellaSwag, WinoGrande, BoolQ, and OpenBookQA.

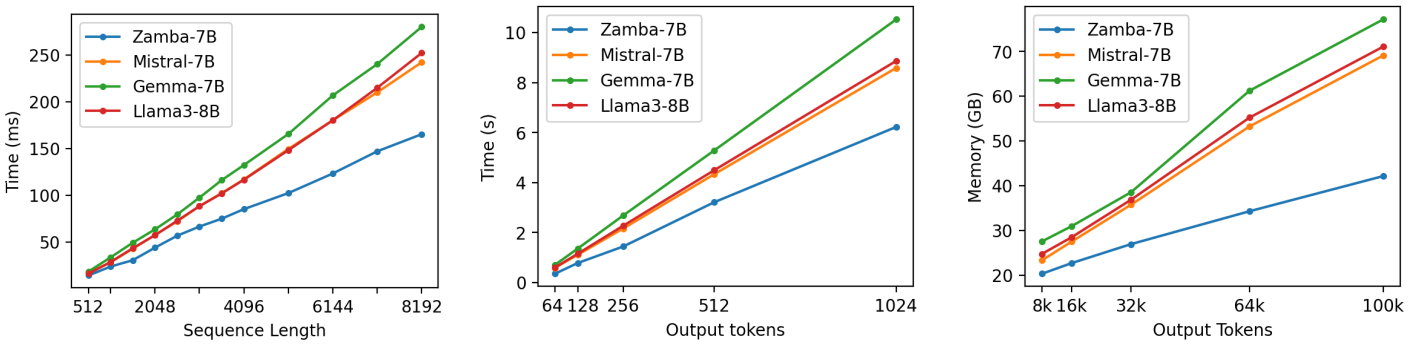
At the time of writing, Zamba is the best-performing open-checkpoints model available. We believe that Zamba is of significant interest to those studying training dynamics, both because of its performance and because of its unique architecture. By comparing representational dynamics in models such as Zamba with standard transformer architectures such as OLMo (Groeneveld et al., 2024)*, we hope that progress will be made towards understanding how model architectures affect the formation of representations during training.

IV. RELATED WORK

A. State-space-models

The quadratic cost of attention in the sequence length has brought about a number of approaches to try to perform

*We were unable to collect evaluation results for OLMo 1.0 MMLU, nor any results for OLMo 1.7 (AI2, 2024), due to incompatibilities with the language model evaluation harness (Gao et al., 2023) v0.4.0.



(a) Inference latency. (b) Generation latency. Input sequence length of 2048. (c) Generation per-GPU memory. 8192 input tokens.

Fig. 4. Inference/generation results on 2 H100 GPUs using tensor parallelism. All numbers taken with a batch size of 1. Competing models were evaluated using vLLM (Kwon et al., 2023).

language modeling capabilities of transformers but without this limitation. If this is possible it would enable significantly larger context lengths and much more efficient generation in terms of both FLOPs and memory required to store the KV cache. One line of work aiming to tackle this problem has been utilizing state-space models (Gu et al., 2021a, 2020, 2021b). State space models utilize a linear dynamical system approach to model language. Such a dynamical system maintains a constant size 'memory' which is maintained throughout the sequence which must encode all the information needed to predict the next token. Unlike RNNs however due to the linearity of the dynamics, SSMs can be written and computed as a convolution or a parallel scan enabling efficient forwarding in a long sequence like a transformer which enables training at scale. The key limitation of SSMs is the restricted form that the sequence mixing can take which enables efficient computation, as well as the compression of all information into a fixed size memory. This renders the expressive power of SSMs less than transformers in theory although it is unclear to what extent natural language requires the full expressive powers that transformers give. While early SSM models performed significantly worse than transformers recent models such as Mamba, and S6 have claimed to perform and scale on par, as have other SSM adjacent architecture such as Griffin (De et al., 2024), RWKV (Peng et al., 2023, 2024), and RetNets (Sun et al., 2023). These models are typically more expressive because they possess selective (input-dependent) gating and control of their internal dynamics, similar to the input dependence of the query, key and value vectors in a transformer.

Recent work however has highlighted potential deficiencies of pure SSMs at in context learning and other algorithmic tasks and several groups have found that hybridizing SSMs with attention gives performance on par or exceeding transformers cite. While most of this work adds many attention layers, with Zamba we pose and show evidence for an interesting hypothesis — that one attention layer is all you need.

B. Annealing

While earlier works may have utilized a two-phase annealing schedule, but not disclosed it, the first work that openly discussed in detail a two-phase schedule was miniCPM (Hu et al., 2024). They describe performing a first slow learning rate decay over the majority of pretraining data, followed by a more rapid decay over high-quality datasets, including instruct data. Since then, several other recently released models have explicitly or elliptically discussed annealing phases. These include Nemotron (Parmar et al., 2024b) and JetMoE (Shen et al., 2024), both claiming that annealing significantly improved model quality. After Zamba's initial release, OLMo 1.7 (AI2, 2024) also claimed to have performed annealing, which significantly improves the results on MMLU (Hendrycks et al., 2021), and utilized a linear scheduler based on a number of higher-quality dataset, such as wikipedia and flan-instruct. Zamba follows a similar approach, but utilizes a different dataset mix, as well as a faster exponential decay schedule and a learning-rate rewarmup from zero.

C. Open checkpoints

While releasing models with full checkpoints during training is rare, there are a number of works that have done so. The Pythia suite (Biderman et al., 2023) pioneered this approach and has been instrumental in many works on interpretability and training dynamics. However, trained for scientific purposes, Pythia models attain far from state of the art performance. Several more recent works also aim to open and democratize LLM pretraining by offering checkpoints and training details. These include OLMo (Groeneveld et al., 2024), who provide detailed training code and checkpoints for a 7B parameter model, and LLM360 (Liu et al., 2023), who provide descriptions of training and open-checkpoints models based on the Llama architecture. Additionally, for study of pure Mamba architectures, Zyphra released open checkpoints of a small 370m pure Mamba model trained on the Pile dataset (Zyphra, 2024).

V. Discussion

In this technical report, we introduce Zamba, a 7B open-source SSM highly competitive with leading models. We thus demonstrate conclusively the scalability of SSM architectures to this scale. Moreover, we utilize a novel architecture using

a shared global attention block which obtains the benefits of hybrid SSM-attention architectures while minimizing the parameters dedicated to attention. Moreover, we describe the two-phase training regime Zamba underwent and release both phase 1 (pretrained) and final (annealed) models. Zamba was trained on a relatively small budget of approximately $200k and a team of 7 researchers over the course of a month and approaches leading models in performance, thus demonstrating that approaching the state-of-the-art in LLM pretraining does not necessarily require vast budgets or teams and is not restricted only to a few leading companies.

It is an interesting question what the source of the remaining gap between Zamba and the leading models at this scale is. The Zamba model trained until the end of phase 1 achieves Llama2 levels of performance from only 1T tokens, while Llama2 was trained on at least 2T tokens. This difference could arise from dataset differences, although given that Zamba's dataset is simply comprised of deduplicated open web datasets, it would be unlikely for Zamba's dataset to be significantly superior in quality to Llama2. It is also possible that our architecture gives us significant advantages over the Llama transformer architecture and indeed, despite claims that Mamba models struggle with ICL (Park et al., 2024), even our base model performs comparably to Llama2 here on MMLU, an evaluation metric which is known to require significant ICL to perform well. It is thus possible that even a single attention layer is enough to reach transformer parity on ICL.

There is then the question of how to close the gap between Zamba and the leading open-weight models such as Mistral, Gemma, and Llama3. It is likely that some fraction of this gap is caused by a large disparity in the number of pretraining tokens. Our model is trained on only 1T tokens vs 8T for Gemma, 15T for Llama3, and an unknown (but likely similarly large) quantity for Mistral. Like others, and in accordance with the Chinchilla scaling laws, we observe continuing increases in performance (on a log scale) even towards the end of training, implying that our model has not plateaued in loss and could usefully be trained on many more tokens. Pretraining dataset quality is likely also a significant factor given that for our dataset, while utilizing existing well-known web datasets, we performed only straightforward filtering and deduplication, while there are many techniques for significantly improved dataset preparation available in the literature (Xie et al., 2024; Tirumala et al., 2024; Maini et al., 2024).

We see significant improvements on many evaluation scores during the annealing phase, providing an independent replication of the claims of miniCPM (Hu et al., 2024). Given the significant jumps between base pretrained models such as Llama1, Llama2, and OLMo with relatively well-known datasets, and leading models in the 7B range such as Mistral and Gemma, it is likely that the augmentation of web data with synthetic, instruct, or other high quality data-sources played a key part in the performance of these models. This could either be during pretraining itself, as argued for by the phi series of models (Li et al., 2023; Abdin et al., 2024), or in an annealing phase similar to miniCPM and the one we performed. Empirically, we see that Zamba's annealing phase closes about half or more of the gap from Llama2 to state-of-the-art models especially on more reasoning-like evaluations such as MMLU (Hendrycks et al., 2021) and ARC (Clark et al., 2018). It is possible that an improved annealing phase on significantly more synthetic data, or pretraining on significantly more tokens may close this gap.

While several recent works have claimed to perform annealing, relatively little has yet been published on the exact methods used for this or the performance of the base model before annealing began. We demonstrate that annealing on high quality data can significantly improve a model from approximately a Llama2 level base to closing the gap with leading models. Understanding the extent to which this performance improvement is real and enhances the model's true capabilities or whether it is simply training the model to respond in a more evaluation-friendly manner remains to be elucidated. Moreover, many questions remain open such as the optimal annealing schedule, the optimal replay fraction of original pretraining data, and the optimal composition of the annealing datasets. We hope that by releasing our base model, and all annealing checkpoints, we can help the academic community begin answering these questions.

With Zamba, by both validating the performance of Mamba at scale and by pioneering a novel architecture beyond that, we have also taken a step towards moving away from the standard transformer architecture for training state-of-the-art models. We believe that architectural innovations are relatively understudied given the success of the scaling paradigm. However, as the cost of training state-of-the-art models increases, and as the benefits of scaling data at the small scale become increasingly marginal, the possibility of constant gains from improved architectures and pretraining paradigms vs logarithmic ones from additional data become increasingly important in pushing forward the frontier in performance for smaller model size brackets.

Finally, with Zamba, we make available all checkpoints during both the pretraining and annealing phase (one every 2500 steps) for scientific study. We believe that many scientific questions of deep interest for understanding the learning dynamics of such models can only be approached by studying the evolution of model weights during training and few academic groups have the budget or expertise to train models close to the state of the art. As such, we encourage other groups who train models at the frontier to consider making available and open the checkpoints of their models as well.

AUTHOR CONTRIBUTIONS

Paolo — Contributed to core infrastructure. Lead annealing experiments. Lead evaluations. Lead HuggingFace conversion and release. Contributed to architecture search experiments.

Quentin — Contributed to core infrastructure. Lead infrastructure development and training optimisation. Lead inference optimization. Contributed to cluster management and maintenance. Contributed to evaluations.

Yury — Contributed to core infrastructure. Lead dataset preparation and processing. Contributed to cluster management and maintenance. Contributed to evaluations.

James — Contributed to architecture search experiments. Invented final architecture used in the paper.

Jonathan — Contributed to architecture search experiments.

Adam — Advised overall the project. Co-lead the curriculum approach. Contributed to annealing experiments.

Beren — Overall project lead. Contributed to core infrastructure. Contributed to annealing experiments. Lead creation and processing of annealing datasets. Contributed to architecture search experiments. Contributed to evaluations. Primary author of the technical report.

ACKNOWLEDGEMENTS

We would like to acknowledge the rest of the Zyphra team for their support, including Steven Brook, Nick Alonso, Vasudev Shyam, Anna Golubeva, Tomas Figliolia and Krithik Puthalath for helpful discussions and feedback.

REFERENCES

Abdin, M., Jacobs, S. A., Awan, A. A., Aneja, J., Awadallah, A., Awadalla, H., Bach, N., Bahree, A., Bakhtiari, A., Behl, H., et al. (2024). Phi-3 technical report: A highly capable language model locally on your phone. *arXiv preprint arXiv:2404.14219*.

AI2 (2024). OLMo 1.7–7B: A 24 point improvement on MMLU. https://blog.allenai.org/olmo-1-7-7b-a-24-point-improvement-on-mmlu-92b43f7d269d. Accessed: May 28, 2024.

Anthony, Q., Tokpanov, Y., Glorioso, P., and Millidge, B. (2024). Blackmamba: Mixture of experts for state-space models. *arXiv preprint arXiv:2402.01771*.

Bachmann, G., Anagnostidis, S., and Hofmann, T. (2023). Scaling mlps: A tale of inductive bias. *Advances in Neural Information Processing Systems*, 36.

Bengio, Y., Louradour, J., Collobert, R., and Weston, J. (2009). Curriculum learning. In *Proceedings of the 26th annual international conference on machine learning*, pages 41–48.

Biderman, S., Schoelkopf, H., Anthony, Q. G., Bradley, H., O'Brien, K., Hallahan, E., Khan, M. A., Purohit, S., Prashanth, U. S., Raff, E., et al. (2023). Pythia: A suite for analyzing large language models across training and scaling. In *International Conference on Machine Learning*, pages 2397–2430. PMLR.

Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D. M., Wu, J., Winter, C., Hesse, C., Chen, M., Sigler, E., Litwin, M., Gray, S., Chess, B., Clark, J., Berner, C., McCandlish, S., Radford, A., Sutskever, I., and Amodei, D. (2020). Language models are few-shot learners. *CoRR*, abs/2005.14165.

Clark, P., Cowhey, I., Etzioni, O., Khot, T., Sabharwal, A., Schoenick, C., and Tafjord, O. (2018). Think you have solved question answering? try arc, the ai2 reasoning challenge.

Dao, T. (2023). Flashattention-2: Faster attention with better parallelism and work partitioning.

De, S., Smith, S. L., Fernando, A., Botev, A., Cristian-Muraru, G., Gu, A., Haroun, R., Berrada, L., Chen, Y., Srinivasan, S., Desjardins, G., Doucet, A., Budden, D., Teh, Y. W., Pascanu, R., Freitas, N. D., and Gulcehre, C. (2024). Griffin: Mixing gated linear recurrences with local attention for efficient language models.

Dehghani, M., Gouws, S., Vinyals, O., Uszkoreit, J., and Kaiser, Ł. (2018). Universal transformers. *arXiv preprint arXiv:1807.03819*.

Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., Dehghani, M., Minderer, M., Heigold, G., Gelly, S., Uszkoreit, J., and Houlsby, N. (2020). An image is worth 16x16 words: Transformers for image recognition at scale. *CoRR*, abs/2010.11929.

Elman, J. L. (1993). Learning and development in neural networks: The importance of starting small. *Cognition*, 48(1):71–99.

Fedus, W., Zoph, B., and Shazeer, N. (2022). Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39.

Gao, L., Biderman, S., Black, S., Golding, L., Hoppe, T., Foster, C., Phang, J., He, H., Thite, A., Nabeshima, N., et al. (2020). The pile: An 800gb dataset of diverse text for language modeling. *arXiv preprint arXiv:2101.00027*.

Gao, L., Tow, J., Abbasi, B., Biderman, S., Black, S., DiPofi, A., Foster, C., Golding, L., Hsu, J., Le Noac'h, A., Li, H., McDonell, K., Muennighoff, N., Ociepa, C., Phang, J., Reynolds, L., Schoelkopf, H., Skowron, A., Sutawika, L., Tang, E., Thite, A., Wang, B., Wang, K., and Zou, A. (2023). A framework for few-shot language model evaluation.

Gemma Team, Mesnard, T., Hardin, C., Dadashi, R., Bhupatiraju, S., Pathak, S., Sifre, L., Rivière, M., Kale, M. S., Love, J., et al. (2024). Gemma: Open models based on gemini research and technology. *arXiv preprint arXiv:2403.08295*.

Grazzi, R., Siems, J., Schrodi, S., Brox, T., and Hutter, F. (2024). Is mamba capable of in-context learning?

Groeneveld, D., Beltagy, I., Walsh, P., Bhagia, A., Kinney, R., Tafjord, O., Jha, A. H., Ivison, H., Magnusson, I., Wang, Y., et al. (2024). Olmo: Accelerating the science of language models. *arXiv preprint arXiv:2402.00838*.

Gu, A. and Dao, T. (2023). Mamba: Linear-time sequence

modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*.

Gu, A., Dao, T., Ermon, S., Rudra, A., and Ré, C. (2020). Hippo: Recurrent memory with optimal polynomial projections. *Advances in neural information processing systems*, 33:1474–1487.

Gu, A., Goel, K., and Ré, C. (2021a). Efficiently modeling long sequences with structured state spaces. *arXiv preprint arXiv:2111.00396*.

Gu, A., Johnson, I., Goel, K., Saab, K., Dao, T., Rudra, A., and Ré, C. (2021b). Combining recurrent, convolutional, and continuous-time models with linear state space layers. *Advances in neural information processing systems*, 34:572–585.

Gunasekar, S., Zhang, Y., Aneja, J., Mendes, C. C. T., Del Giorno, A., Gopi, S., Javaheripi, M., Kauffmann, P., de Rosa, G., Saarikivi, O., et al. (2023). Textbooks are all you need. *arXiv preprint arXiv:2306.11644*.

Gupta, K., Thérien, B., Ibrahim, A., Richter, M. L., Anthony, Q., Belilovsky, E., Rish, I., and Lesort, T. (2023). Continual pre-training of large language models: How to (re) warm your model? *arXiv preprint arXiv:2308.04014*.

Hendrycks, D., Burns, C., Basart, S., Zou, A., Mazeika, M., Song, D., and Steinhardt, J. (2021). Measuring massive multitask language understanding.

Hoffmann, J., Borgeaud, S., Mensch, A., Buchatskaya, E., Cai, T., Rutherford, E., de Las Casas, D., Hendricks, L. A., Welbl, J., Clark, A., Hennigan, T., Noland, E., Millican, K., van den Driessche, G., Damoc, B., Guy, A., Osindero, S., Simonyan, K., Elsen, E., Rae, J. W., Vinyals, O., and Sifre, L. (2022). Training compute-optimal large language models.

Hu, S., Tu, Y., Han, X., He, C., Cui, G., Long, X., Zheng, Z., Fang, Y., Huang, Y., Zhao, W., et al. (2024). Minicpm: Unveiling the potential of small language models with scalable training strategies. *arXiv preprint arXiv:2404.06395*.

Ibrahim, A., Thérien, B., Gupta, K., Richter, M. L., Anthony, Q., Lesort, T., Belilovsky, E., and Rish, I. (2024). Simple and scalable strategies to continually pre-train large language models. *arXiv preprint arXiv:2403.08763*.

Jacobs, R. A., Jordan, M. I., Nowlan, S. J., and Hinton, G. E. (1991). Adaptive mixtures of local experts. *Neural computation*, 3(1):79–87.

Jelassi, S., Brandfonbrener, D., Kakade, S. M., and Malach, E. (2024). Repeat after me: Transformers are better than state space models at copying.

Kaplan, J., McCandlish, S., Henighan, T., Brown, T. B., Chess, B., Child, R., Gray, S., Radford, A., Wu, J., and Amodei, D. (2020). Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*.

Krishna, K., Garg, S., Bigham, J. P., and Lipton, Z. (2023). Downstream datasets make surprisingly good pretraining corpora. In *The 61st Annual Meeting Of The Association For Computational Linguistics*.

Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., and Stoica, I. (2023). Efficient memory management for large language model serving with pagedattention. In *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*.

Li, Y., Bubeck, S., Eldan, R., Del Giorno, A., Gunasekar, S., and Lee, Y. T. (2023). Textbooks are all you need ii: phi-1.5 technical report. *arXiv preprint arXiv:2309.05463*.

Lieber, O., Lenz, B., Bata, H., Cohen, G., Osin, J., Dalmedigos, I., Safahi, E., Meirom, S., Belinkov, Y., Shalev-Shwartz, S., Abend, O., Alon, R., Asida, T., Bergman, A., Glozman, R., Gokhman, M., Manevich, A., Ratner, N., Rozen, N., Shwartz, E., Zusman, M., and Shoham, Y. (2024). Jamba: A hybrid transformer-mamba language model.

Liu, C., He, S., Liu, K., and Zhao, J. (2018). Curriculum learning for natural answer generation. In *Proceedings of the 27th International Joint Conference on Artificial Intelligence*, pages 4223–4229.

Liu, Z., Qiao, A., Neiswanger, W., Wang, H., Tan, B., Tao, T., Li, J., Wang, Y., Sun, S., Pangarkar, O., et al. (2023). Llm360: Towards fully transparent open-source llms. *arXiv preprint arXiv:2312.06550*.

Maini, P., Seto, S., Bai, H., Grangier, D., Zhang, Y., and Jaitly, N. (2024). Rephrasing the web: A recipe for compute and data-efficient language modeling. *arXiv preprint arXiv:2401.16380*.

NVIDIA (2023). Transformer Engine: A library for accelerating Transformer models on NVIDIA GPUs. https://github.com/NVIDIA/TransformerEngine. Accessed: May 28, 2024.

Park, J., Park, J., Xiong, Z., Lee, N., Cho, J., Oymak, S., Lee, K., and Papailiopoulos, D. (2024). Can mamba learn how to learn? a comparative study on in-context learning tasks.

Parmar, J., Prabhumoye, S., Jennings, J., Patwary, M., Subramanian, S., Su, D., Zhu, C., Narayanan, D., Jhunjhunwala, A., Dattagupta, A., et al. (2024a). Nemotron-4 15b technical report. *arXiv preprint arXiv:2402.16819*.

Parmar, J., Prabhumoye, S., Jennings, J., Patwary, M., Subramanian, S., Su, D., Zhu, C., Narayanan, D., Jhunjhunwala, A., Dattagupta, A., Jawa, V., Liu, J., Mahabaleshwarkar, A., Nitski, O., Brundyn, A., Maki, J., Martinez, M., You, J., Kamalu, J., LeGresley, P., Fridman, D., Casper, J., Aithal, A., Kuchaiev, O., Shoeybi, M., Cohen, J., and Catanzaro, B. (2024b). Nemotron-4 15b technical report.

Penedo, G., Malartic, Q., Hesslow, D., Cojocaru, R., Cappelli, A., Alobeidli, H., Pannier, B., Almazrouei, E., and Launay, J. (2023). The refinedweb dataset for falcon llm: outperforming curated corpora with web data, and web data only. *arXiv preprint arXiv:2306.01116*.

Peng, B., Alcaide, E., Anthony, Q., Albalak, A., Arcadinho, S., Biderman, S., Cao, H., Cheng, X., Chung, M., Grella, M., GV, K. K., He, X., Hou, H., Lin, J., Kazienko, P., Kocon, J., Kong, J., Koptyra, B., Lau, H., Mantri, K. S. I., Mom, F., Saito, A., Song, G., Tang, X., Wang, B., Wind, J. S., Wozniak, S., Zhang, R., Zhang, Z., Zhao, Q., Zhou, P., Zhou, Q., Zhu, J., and Zhu, R.-J. (2023). Rwkv: Reinventing rnns for the transformer era.

Peng, B., Goldstein, D., Anthony, Q., Albalak, A., Alcaide, E., Biderman, S., Cheah, E., Du, X., Ferdinan, T., Hou, H., Kazienko, P., GV, K. K., Kocoń, J., Koptyra, B., Krishna, S., au2, R. M. J., Muennighoff, N., Obeid, F., Saito, A., Song, G., Tu, H., Woźniak, S., Zhang, R., Zhao, B., Zhao, Q., Zhou, P., Zhu, J., and Zhu, R.-J. (2024). Eagle and finch: Rwkv with matrix-valued states and dynamic recurrence.

Poli, M., Thomas, A. W., Nguyen, E., Ponnusamy, P., Deiseroth, B., Kersting, K., Suzuki, T., Hie, B., Ermon, S., Ré, C., et al. (2024). Mechanistic design and scaling of hybrid architectures. *arXiv preprint arXiv:2403.17844*.

Rae, J. W., Borgeaud, S., Cai, T., Millican, K., Hoffmann, J., Song, F., Aslanides, J., Henderson, S., Ring, R., Young, S., Rutherford, E., Hennigan, T., Menick, J., Cassirer, A., Powell, R., van den Driessche, G., Hendricks, L. A., Rauh, M., Huang, P.-S., Glaese, A., Welbl, J., Dathathri, S., Huang, S., Uesato, J., Mellor, J., Higgins, I., Creswell, A., McAleese, N., Wu, A., Elsen, E., Jayakumar, S., Buchatskaya, E., Budden, D., Sutherland, E., Simonyan, K., Paganini, M., Sifre, L., Martens, L., Li, X. L., Kuncoro, A., Nematzadeh, A., Gribovskaya, E., Donato, D., Lazaridou, A., Mensch, A., Lespiau, J.-B., Tsimpoukelli, M., Grigorev, N., Fritz, D., Sottiaux, T., Pajarskas, M., Pohlen, T., Gong, Z., Toyama, D., de Masson d'Autume, C., Li, Y., Terzi, T., Mikulik, V., Babuschkin, I., Clark, A., de Las Casas, D., Guy, A., Jones, C., Bradbury, J., Johnson, M., Hechtman, B., Weidinger, L., Gabriel, I., Isaac, W., Lockhart, E., Osindero, S., Rimell, L., Dyer, C., Vinyals, O., Ayoub, K., Stanway, J., Bennett, L., Hassabis, D., Kavukcuoglu, K., and Irving, G. (2022). Scaling language models: Methods, analysis & insights from training gopher.

Raffel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., Zhou, Y., Li, W., and Liu, P. J. (2020). Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of machine learning research*, 21(140):1–67.

Rajbhandari, S., Li, C., Yao, Z., Zhang, M., Aminabadi, R. Y., Awan, A. A., Rasley, J., and He, Y. (2022). Deepspeed-moe: Advancing mixture-of-experts inference and training to power next-generation ai scale. In *International conference on machine learning*, pages 18332–18346. PMLR.

Shazeer, N., Mirhoseini, A., Maziarz, K., Davis, A., Le, Q., Hinton, G., and Dean, J. (2016). Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *International Conference on Learning Representations*.

Shen, Y., Guo, Z., Cai, T., and Qin, Z. (2024). Jetmoe: Reaching llama2 performance with 0.1 m dollars. *arXiv preprint arXiv:2404.07413*.

Soldaini, L. and Lo, K. (2023). peS2o (Pretraining Efficiently on S2ORC) Dataset. Technical report, Allen Institute for AI. ODC-By, https://github.com/allenai/pes2o.

Sun, Y., Dong, L., Huang, S., Ma, S., Xia, Y., Xue, J., Wang, J., and Wei, F. (2023). Retentive network: A successor to transformer for large language models.

Team, G., Anil, R., Borgeaud, S., Wu, Y., Alayrac, J.-B., Yu, J., Soricut, R., Schalkwyk, J., Dai, A. M., Hauth, A., et al. (2023). Gemini: a family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805*.

Tirumala, K., Simig, D., Aghajanyan, A., and Morcos, A. (2024). D4: Improving llm pretraining via document de-duplication and diversification. *Advances in Neural Information Processing Systems*, 36.

Touvron, H., Martin, L., Stone, K., Albert, P., Almahairi, A., Babaei, Y., Bashlykov, N., Batra, S., Bhargava, P., Bhosale, S., et al. (2023). Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*.

Tscshantz, A., Millidge, B., Seth, A. K., and Buckley, C. L. (2023). Hybrid predictive coding: Inferring, fast and slow. *PLoS Computational Biology*, 19(8):e1011280.

van Bergen, R. S. and Kriegeskorte, N. (2020). Going in circles is the way forward: the role of recurrence in visual inference. *Current Opinion in Neurobiology*, 65:176–193.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. (2017). Attention is all you need. *Advances in neural information processing systems*, 30.

Whittington, J. C. R., Muller, T. H., Mark, S., Barry, C., Burgess, N., Behrens, T. E. E., Chen, G., Barry, C., Burgess, N., and Behrens, T. E. E. (2020). The Tolman-Eichenbaum Machine: Unifying Space and Relational Memory through Generalization in the Hippocampal Formation. *Cell*, 183(5):1249–1263.e23. Publisher: Elsevier Inc.

Whittington, J. C. R., Warren, J., and Behrens, T. E. J. (2021). Relating transformers to models and neural representations of the hippocampal formation. *International Conference on Learning Representations*.

Xie, S. M., Pham, H., Dong, X., Du, N., Liu, H., Lu, Y., Liang, P. S., Le, Q. V., Ma, T., and Yu, A. W. (2024). Doremi: Optimizing data mixtures speeds up language model pretraining. *Advances in Neural Information Processing Systems*, 36.

Yang, Z., Zeng, X., Zhao, Y., and Chen, R. (2023). Alphafold2 and its applications in the fields of biology and medicine. *Signal Transduction and Targeted Therapy*, 8(1):115.

Zyphra (2024). Reproduction of Mamba-370M by Zyphra. https://huggingface.co/Zyphra/Mamba-370M. Accessed: May 28, 2024.

APPENDIX

MODEL AND TRAINING HYPERPARAMETERS

Hyperparameter	Value
Number of Layers	80
Hidden Dimension	3712
State Dimension	16
Convolution Dimension	4
Number of Attention Heads	16
Context Length	4096
Batch Size	512
Max Learning Rage	1.5e-4
LR Decay Schedule	Cosine
Minimum LR	7.5e-5
Weight Decay	0.1
Adam Beta2	0.95
LR Warmup	0.01
Gradient Clipping	1.0
Training Precision	BF16